

MANUAL TÉCNICO

1. Descripción del Proyecto

Este proyecto implementa las fases de análisis léxico y sintáctico de un compilador para GoLite, utilizando herramientas de generación automática de analizadores. El objetivo es proporcionar una base sólida para la traducción y ejecución del lenguaje.

2. Arquitectura y Herramientas

El sistema fue desarrollado bajo los siguientes componentes tecnológicos:

- Lenguaje base: Java (JDK 21).
- Generador de Analizador Léxico: JFlex 1.9.1 (Encargado de convertir las expresiones regulares en un autómata finito determinista para el reconocimiento de tokens).
- Generador de Analizador Sintáctico: CUP (v11b) (Encargado de procesar la gramática libre de contexto y generar el árbol de derivación).

3. Diseño del Analizador Léxico (JFlex)

El analizador léxico (GoLiteLexer.jflex) se basa en una tabla de transiciones definida a partir de expresiones regulares que capturan:

- Palabras reservadas: (if, for, var, func, int, bool, etc.).
- Operadores aritméticos y lógicos: (+, -, *, /, &&, ||, !).
- Símbolos y delimitadores: ({, }, (,), ;, ,).
- Literales: Manejo de enteros, flotantes, cadenas y runas.

4. Diseño del Analizador Sintáctico (CUP)

La gramática (GoLiteParser.cup) define las reglas de producción para la estructura del lenguaje. Los componentes principales incluyen:

- Producciones principales: Definición de programas, listas de sentencias, bloques de código y estructuras de control.
- Precedencia: Se han definido niveles de precedencia para los operadores aritméticos y lógicos para evitar ambigüedades.
- Manejo de ámbitos: El parser está diseñado para reconocer la jerarquía de los bloques {} para el manejo correcto de variables (shadowing).

5. Manejo de Errores

El sistema implementa una clase ReportCollector que centraliza los errores encontrados durante el proceso de compilación:

- **Error recuperable:** El analizador léxico reporta caracteres no reconocidos pero continúa el escaneo para encontrar todos los errores del archivo.
- **Error sintáctico:** El analizador sintáctico detiene o reporta la estructura mal formada basándose en el símbolo que no pudo reducir según la gramática.

4. Gramática del Lenguaje (Representación en Texto)

La estructura sintáctica de GoLite se define a través de las siguientes reglas de producción, que dictan cómo se organizan los tokens para formar sentencias válidas:

4.1. Estructura General

- **Programa:** Programa -> ListaSentencias
- **ListaSentencias:** ListaSentencias -> Sentencia | ListaSentencias Sentencia

4.2. Declaración y Asignación

- **Declaración de Variable:** VarDecl -> VAR IDENTIFICADOR TIPO = Expresion ;
- **Asignación:** Asignacion -> IDENTIFICADOR = Expresion ;
- **Tipo:** Tipo -> INT | BOOL | FLOAT64 | STRING

4.3. Estructuras de Control

- **Sentencia IF:** IfStmt -> IF Expresion { ListaSentencias } IfStmt -> IF Expresion { ListaSentencias } ELSE { ListaSentencias }
- **Sentencia FOR:** ForStmt -> FOR Expresion ; Expresion ; Expresion { ListaSentencias } (*Nota: Incluye el formato FOR ; Expresion ; Expresion { ... } para iteración clásica*)

4.4. Expresiones y Otros

- **Sentencia Print:** PrintStmt -> FMT.PRINTLN (Expresion) ;
- **Bloque:** Bloque -> { ListaSentencias }
- **Expresión:** Expresion -> Termino Expresion -> Expresion Operador Expresion
- **Operadores:**
 - Aritméticos: +, -, *, /, %
 - Relacionales: ==, !=, <, >, <=, >=

- Lógicos: &&, ||, !